

# Module 2: Variables, Data Types, and Constants

## 2.1 What is a Variable

- A variable is a named storage location in the computer's memory that holds a value
- Think of it as a labeled box that can store one piece of information at a time
- The value inside a variable can be changed during program execution
- Every variable has three important properties
  - Name (identifier): used to refer to the variable in code
  - Data type: determines what kind of value it can hold and how much memory it uses
  - Value: the actual data stored at that moment

## 2.2 Declaring and Initializing Variables

- Declaration: telling the compiler that a variable with a given name and type exists
  - Syntax: `data_type variable_name;`
  - Example: `int age;`
- Initialization: assigning a value to a variable at the time of declaration
  - Syntax: `data_type variable_name = value;`
  - Example: `int age = 20;`
- Assignment after declaration: giving a value to a variable after it is declared
  - Example: `age = 25;`
- Declaring multiple variables of the same type on one line
  - Example: `int x = 5, y = 10, z = 15;`
- What happens if a variable is used before it is initialized
  - It contains a garbage value (random leftover data in memory)
  - Always initialize variables before using them

## 2.3 Rules for Naming Variables

- Variable names can contain letters, digits, and underscores
- The name must start with a letter or an underscore, not a digit
- Variable names cannot contain spaces or special symbols like @, #, or \$
- C++ keywords (reserved words) cannot be used as variable names
- Examples of keywords: `int`, `float`, `return`, `if`, `while`, `for`, `class`
- Variable names are case-sensitive
- `score`, `Score`, and `SCORE` are three different variables
- Good naming habits
- Use descriptive names that explain what the variable stores
- Use camelCase (`studentAge`) or underscore\_separated (`student_age`) consistently

- Avoid single-letter names except for simple loop counters like i, j, k
- Valid variable names: age, student\_name, totalScore, x1  
 Invalid variable names: 1age, my-name, float, total score

## 2.4 Fundamental Data Types

Integer types (whole numbers without a decimal point)

- int
- Standard integer type
- Typically 4 bytes (32 bits) on most systems
- Range: -2,147,483,648 to 2,147,483,647
- Example: int score = 95;
- short int (or just short)
- Uses 2 bytes (16 bits)
- Range: -32,768 to 32,767
- Used when memory is very limited
- long int (or just long)
- 4 or 8 bytes depending on the system
- Larger range than int
- long long int (or just long long)
- 8 bytes (64 bits)
- Range: -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
- Example: long long bigNumber = 9000000000;

Floating point types (numbers with a decimal point)

- float
- 4 bytes, single precision
- Stores up to approximately 7 significant decimal digits
- Example: float price = 19.99f; (the f suffix marks it as float)
- double
- 8 bytes, double precision
- Stores up to approximately 15 significant decimal digits
- Default type for decimal numbers in C++
- Example: double pi = 3.14159265358979;
- long double
- Extended precision, typically 12 or 16 bytes

Character type

- char
  - Stores a single character such as a letter, digit, or symbol
  - Occupies 1 byte
  - Characters are enclosed in single quotes: 'A', '5', '@'
  - Internally stored as an integer using the ASCII code
- 'A' is stored as 65, 'B' as 66, 'a' as 97, '0' as 48
- Example: char grade = 'A';
  - Difference: 'A' is a character (1 byte), "A" is a string (2 bytes including null terminator)

Boolean type

- bool
- Stores only one of two values: true or false

- Internally: true is stored as 1, false is stored as 0
- Example: `bool isPassed = true;`
- Used with conditions and comparisons

Void type

- Represents the absence of a type
- Used for functions that do not return any value
- Cannot declare a variable of type void

## 2.5 Signed and Unsigned Types

- By default, int, short, long, and char are signed (can hold negative and positive values)
- Signed int range on 4 bytes: -2,147,483,648 to 2,147,483,647
- Unsigned modifier allows only non-negative values but doubles the positive range
- unsigned int range: 0 to 4,294,967,295
- unsigned char range: 0 to 255 (instead of -128 to 127)
- Declaration examples
- `unsigned int count = 100;`
- `unsigned char byte = 200;`
- Use unsigned when you are sure the value will never be negative (like a count or size)

## 2.6 Size of Data Types

- The `sizeof()` operator returns the number of bytes a data type or variable uses
  - Example: `sizeof(int)` returns 4 on most systems
  - Why sizes can differ: depends on the operating system and the compiler
  - Always use `sizeof()` rather than assuming a fixed size if portability matters
  - Checking sizes of all types
- ```
cout << sizeof(int) << endl;
cout << sizeof(double) << endl;
cout << sizeof(char) << endl;
```

## 2.7 Constants

- A constant is a value that is set once and never changes during program execution
- Using the `const` keyword
- `const data_type NAME = value;`
- Example: `const double PI = 3.14159;`
- Attempting to change a `const` variable causes a compiler error
- Using the `#define` preprocessor directive
- `#define PI 3.14159`
- The preprocessor replaces every occurrence of PI in the code with 3.14159 before compilation
- No data type, no semicolon at the end of `#define`
- Differences between `const` and `#define`

- `const` has a data type and is checked by the compiler
- `#define` is a simple text substitution with no type checking
- `const` is preferred in modern C++
- Naming convention: constants are typically written in ALL\_CAPS with underscores
- Examples: `MAX_SIZE`, `SPEED_OF_LIGHT`, `TAX_RATE`
- Literal constants in code
- Integer literal: `42`
- Floating point literal: `3.14`
- Character literal: `'Z'`
- String literal: `"Hello"`
- Boolean literal: `true` or `false`

## 2.8 The `auto` Keyword

- Introduced in C++11 to let the compiler automatically deduce the data type
- The compiler looks at the assigned value to figure out the type
- Example: `auto x = 10;` (`x` becomes `int`)
- Example: `auto y = 3.14;` (`y` becomes `double`)
- Useful when the type is long or complex
- Not recommended for beginners when learning types because it hides the data type from the reader